
Internet Of Thinking II

Praktikum Pertemuan 5



Oleh:

NAMA : Bakti Ari Putra

NPM : 25782092

KELAS : TRI – 3D

Dosen:

Dr.Ir.Septafiansyah Dwi Putra, S.T., M.T.

Agus Ambarwari, S.Pd., M.Kom.

**TEKNOLOGI REKAYASA INTERNET
JURUSAN TEKNOLOGI INFORMASI
POLITEKNIK NEGERI LAMPUNG
2026/2027**

Kebutuhan Perangkat Keras

Pastikan komponen berikut tersedia di meja Anda:

- *Board* NodeMCU ESP8266 (Minimal 2 unit, disarankan 3 unit)
- Kabel Micro-USB Data (1 per mahasiswa)
- *Breadboard* / *Project Board* (secukupnya)
- Sensor Suhu & Kelembapan DHT11 atau DHT22 (1 buah per board / mahasiswa)
- Sensor Cahaya LDR (1 buah)
- Resistor 10k Ohm (1 buah, pembagi tegangan LDR)
- LED 5mm (1 buah)
- Resistor 220 Ohm (1 buah, pembatas arus LED)
- Kabel *Jumper* (*Male-to-Male* dan *Male-to-Female* secukupnya)

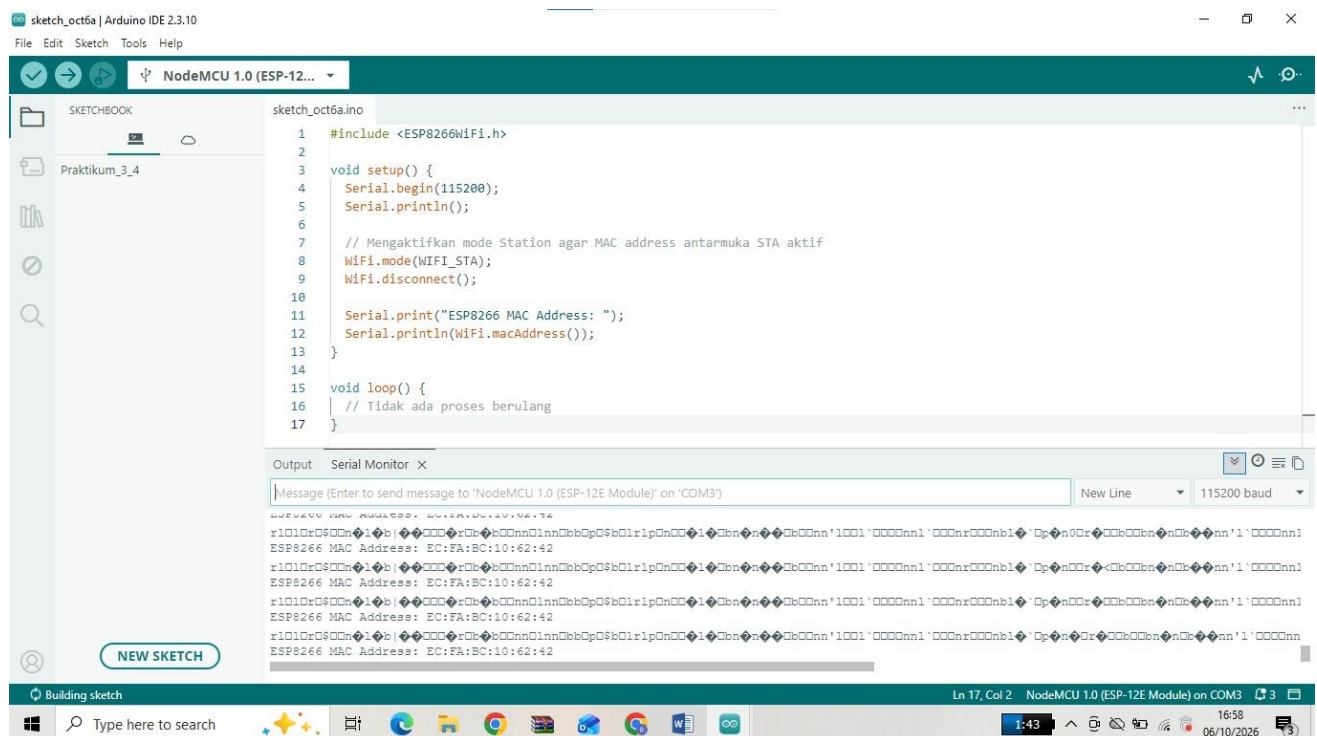
Persiapan Perangkat Lunak & Jaringan

1. Pada Arduino IDE, buka menu **Sketch > Include Library > Manage Libraries...**
2. Ketikkan kata kunci "**painlessMesh**" di kolom pencarian (oleh *Coopdis, Scotty, etc.*).
3. Klik tombol **Install**. Saat muncul jendela konfirmasi dependensi pustaka tambahan, pastikan Anda mengeklik "**Install All**" agar pustaka pendukung berikut terpasang otomatis:
 - ArduinoJson (oleh Benoit Blanchon)
 - TaskScheduler (oleh Anatoli Arkhipenko)
 - ESPAsyncTCP (oleh dvarrel / me-no-dev)

Dokumentasi foto atau video sirkuit/rangkaian yang berhasil diuji.

Praktikum 1: Identifikasi MAC Address Perangkat NodeMCU

Sebelum membangun jaringan ESP-NOW, setiap anggota kelompok wajib membaca dan mencatat alamat fisik (*MAC Address*) dari masing-masing *board* NodeMCU.

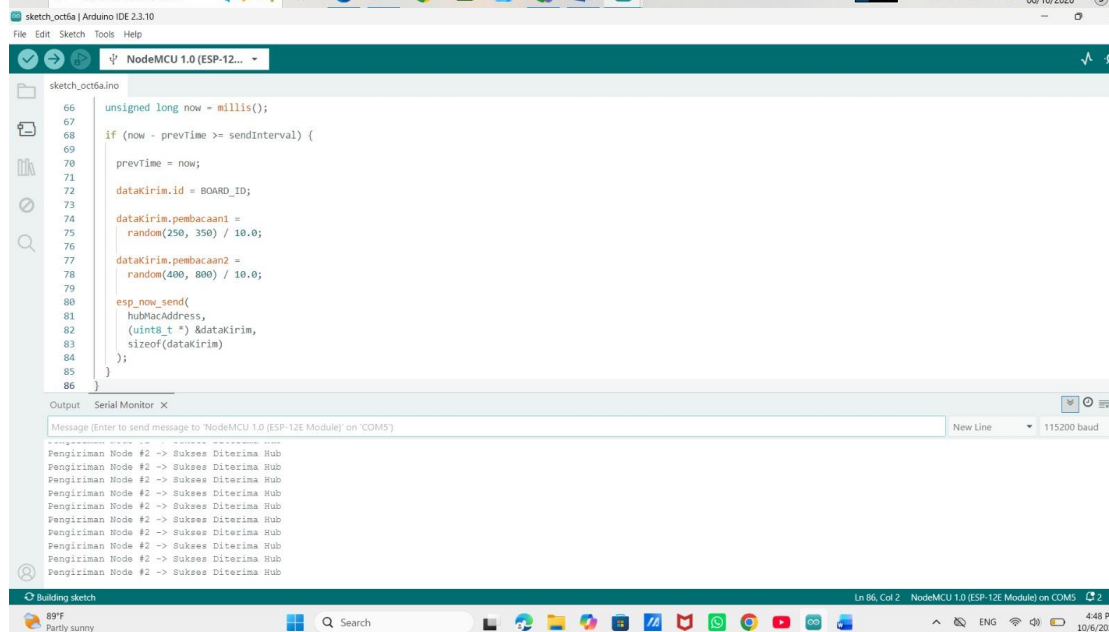
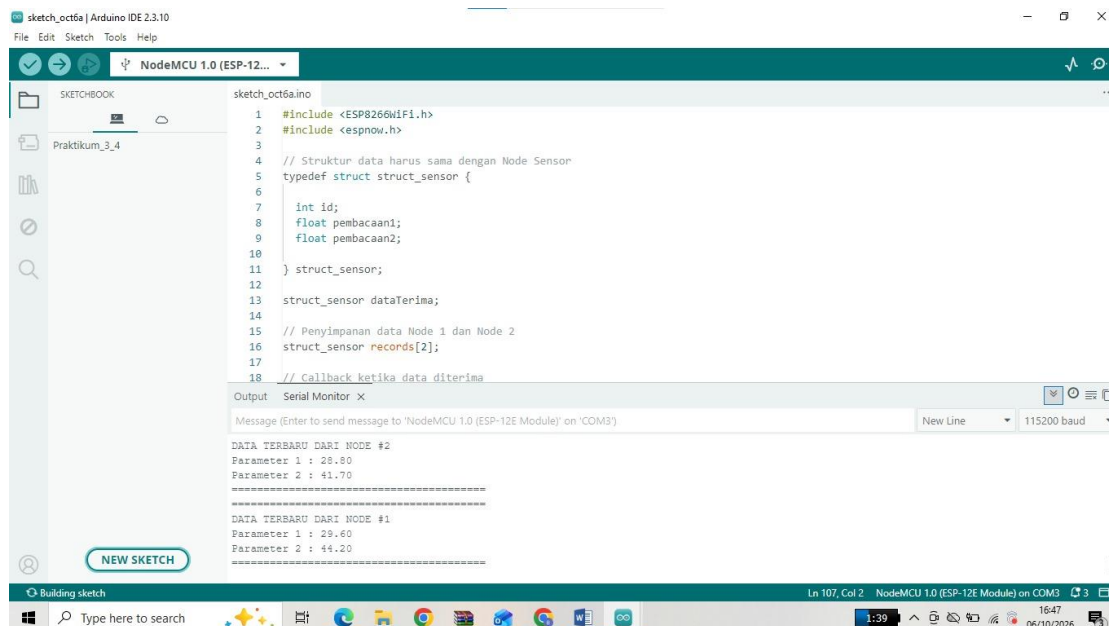
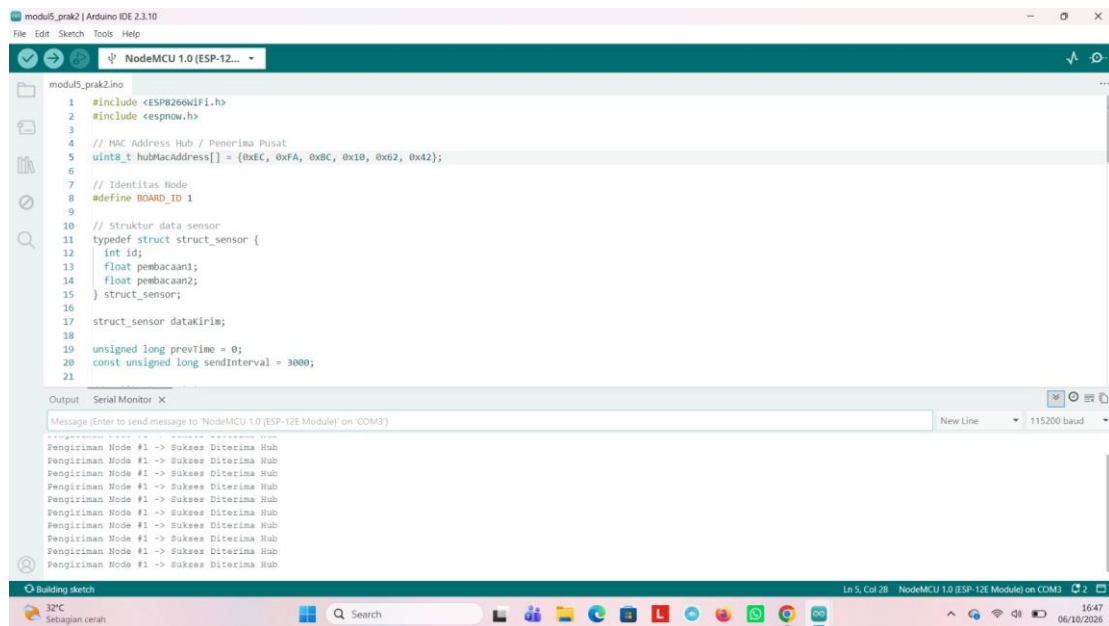


Praktikum 2: Komunikasi Nirkabel One-to-Many (ESP-NOW)

Skenario: Satu unit NodeMCU bertindak sebagai **Controller / Sender**, menyiarkan paket data instruksi yang sama secara langsung ke dua unit NodeMCU lain yang bertindak sebagai **Slave / Receiver** tanpa perantara router.

Langkah Kerja Pemrograman Sisi Pengirim (Controller):

1. Masukkan alamat MAC milik Board Penerima 1 dan Board Penerima 2 ke dalam variabel receiver1 dan receiver2.
2. Format tanda titik dua (:) diubah menjadi bilangan heksadesimal berawalan 0x yang dipisahkan oleh tanda koma (contoh: BC:FF:4D:23:45:90 → {0xBC, 0xFF, 0x4D, 0x23, 0x45, 0x90}).
3. Tuliskan kode berikut dan unggah ke NodeMCU 1:



Praktikum 3: Pengumpulan Data Telemetri Many-to-One (ESP-NOW)

Skenario: Dua node sensor di lapangan memancarkan data pengukuran masing-masing ke sebuah node konsentrator pusat (*Aggregator Hub / Base Station*).

langkah Kerja Pemrograman Sisi Node Pengirim:

1. Masukkan alamat MAC milik **Board Penerima Pusat (Hub)** pada variabel `hubMacAddress`.
2. Pada Board Pengirim 1, atur: `#define BOARD_ID 1`.
3. Pada Board Pengirim 2, atur: `#define BOARD_ID 2`.

```
sketch_oct6a.ino
1 #include <ESP8266WiFi.h>
2 #include <espnow.h>
3
4 // Struktur data penerima (wajib identik dengan sisi pengirim)
5 typedef struct struct_pesan {
6     int perintahId;
7     int nilaiParameter;
8 } struct_pesan;
9
10 struct_pesan paketTerima;
11
12 // Callback otomatis saat antenna mendeteksi paket masuk
13 void OnDataRecv(uint8_t *mac_addr, uint8_t *incomingData, uint8_t len) {
14     char macStr[18];
15     snprintf(macStr, sizeof(macStr), "%02x:%02x:%02x:%02x:%02x:%02x",
16             mac_addr[0], mac_addr[1], mac_addr[2], mac_addr[3], mac_addr[4], mac_addr[5]);
17
18     // Salin buffer memori radio langsung ke struktur data variabel
```

```
modul5_prakt1.ino
1 #include <ESP8266WiFi.h>
2 #include <espnow.h>
3
4 // GANTI dengan MAC Address Board Penerima 1 dan Board Penerima 2 Andai
5 uint8_t receiver1[] = {0xE8, 0x68, 0xE7, 0xE0, 0x9B, 0xED};
6 uint8_t receiver2[] = {0xEC, 0xFA, 0xBC, 0x10, 0x62, 0x42};
7 uint8_t receiver3[] = {0x2C, 0xF4, 0x32, 0x6A, 0x7D, 0x64};
8
9 // Definisi struktur data paket instruksi
10 typedef struct struct_pesan {
11     int perintahId;
12     int nilaiParameter;
13 } struct_pesan;
14
15 struct_pesan paketKirim;
16
17 unsigned long previousMillis = 0;
18 const long interval = 2000; // Transmisi berkala setiap 2 detik
19
20 // Callback otomatis saat paket selesai dipancarkan radio
21 void OnDataSent(uint8_t *mac_addr, uint8_t sendStatus) {
```

Praktikum 4: Jaringan Sensor Ad-Hoc Multi-Hop (painlessMesh)

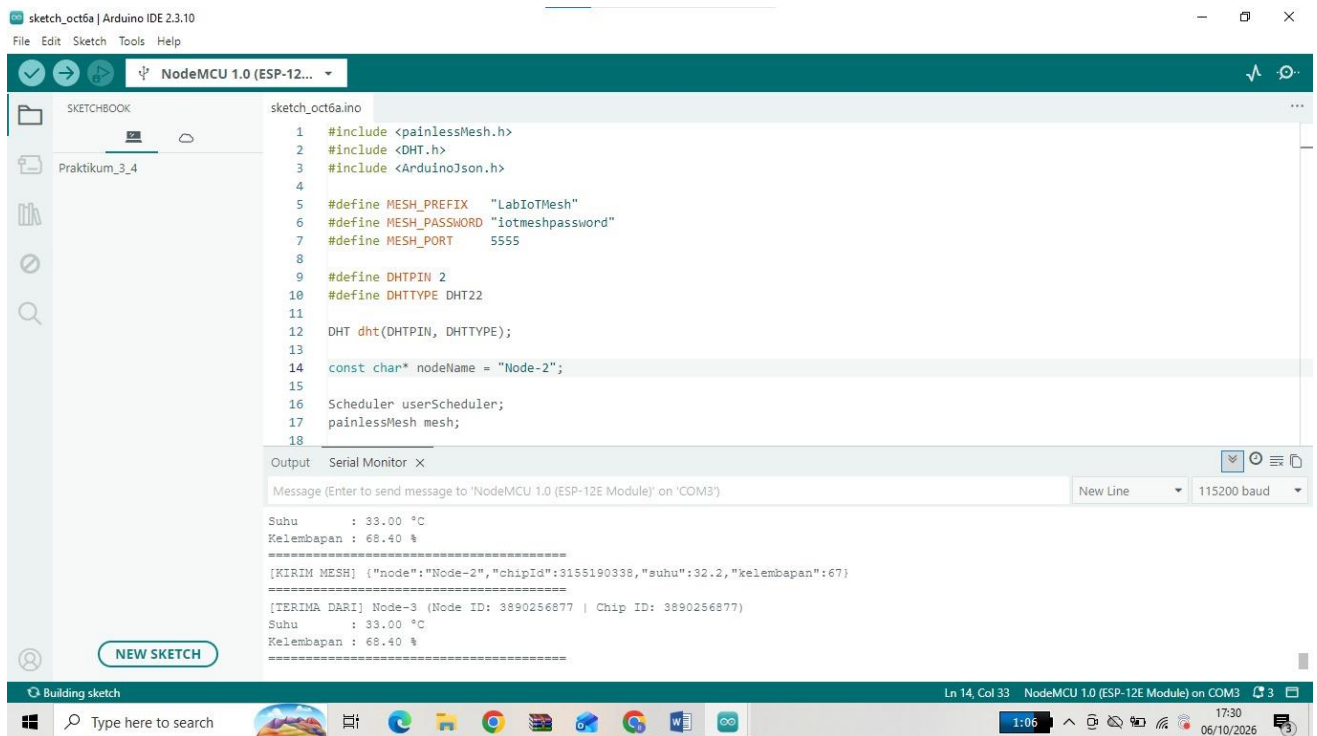
Skenario: Seluruh board dalam kelompok dihubungkan ke sensor suhu & kelembapan riil (DHT11/DHT22) dan diintegrasikan ke dalam sebuah jaringan rantai otonom (*Mesh*). Setiap board secara berkala membaca sensor fisiknya, mengemas data telemetri ke dalam format JSON, dan menyiarkannya ke seluruh anggota jaringan tanpa perlu mengetahui alamat MAC masing-masing.

Instruksi Perangkaian Perangkat Keras:

- Pasang sensor DHT11 / DHT22 pada masing-masing board NodeMCU:
 - Pin VCC sensor ke pin 3V3 (3.3V) NodeMCU.
 - Pin GND sensor ke pin GND NodeMCU.
 - Pin DATA / OUT sensor ke pin D4 (GPIO 2) NodeMCU.

Langkah Kerja Pemrograman:

1. Unggah sketsa berikut ke seluruh board dalam kelompok Anda.
2. Berikan nama pengenalan unik pada variabel nodeName di masing-masing board (misal: "Node-1", "Node-2", "Node-3").
3. Jika menggunakan varian DHT22, ubah #define DHTTYPE DHT11 menjadi DHT22.



```
1 #include <painlessMesh.h>
2 #include <DHT.h>
3 #include <ArduinoJson.h>
4
5 #define MESH_PREFIX "LabIoTMesh"
6 #define MESH_PASSWORD "iotmeshpassword"
7 #define MESH_PORT 5555
8
9 #define DHTPIN 2
10 #define DHTTYPE DHT22
11
12 DHT dht(DHTPIN, DHTTYPE);
13
14 const char* nodeName = "Node-2";
15
16 Scheduler userScheduler;
17 painlessMesh mesh;
18
```

Output Serial Monitor X

Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM3')

Suhu : 33.00 °C
Kelembapan : 68.40 %

[KIRIM MESH] {"node":"Node-2","chipId":3155190338,"suhu":32.2,"kelembapan":67}

[TERIMA DARI] Node-3 (Node ID: 3890256877 | Chip ID: 3890256877)

Suhu : 33.00 °C
Kelembapan : 68.40 %

sketch_sep21a | Arduino IDE 2.3.10

File Edit Sketch Tools Help

NodeMCU 1.0 (ESP-12E Module)

LIBRARY MANAGER

painlessMesh

Type: All

Topic: All

Alteriom PainlessMesh by Coopdis,Scotty...

A painless way to setup a mesh with ESP8266 and ESP32 devices with Alteriom extensions...

More info

2.1.1 INSTALL

Painless Mesh by Coopdis,Scotty...

1.5.7 installed

A painless way to setup a mesh with ESP8266 and ESP32 devices. PainlessMesh is a user-friendly...

More info

1.5.7 REMOVE

sketch_sep21a.ino

```
1 #include <painlessMesh.h>
2 #include <DHT.h>
3 #include <ArduinoJson.h>
4
5 #define MESH_PREFIX "LabIoTMesh"
6 #define MESH_PASSWORD "iotmeshpassword"
7 #define MESH_PORT 5555
8
9 #define DHTPIN 2
10 #define DHTTYPE DHT22
11
12 DHT dht(DHTPIN, DHTTYPE);
13
14 const char* nodeName = "Node-3";
15
16 Scheduler userscheduler;
17 painlessMesh mesh;
18
19 void sendMessage();
20
21 Task taskSendMessage(
22   TASK_SECOND, 3
23 )
```

Output Serial Monitor X

Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM5')

New Line 115200 baud

[TERIMA DARI] Node-2 (Node ID: 3155190338 | Chip ID: 3155190338)
Suhu : 32.20 °C
Kelembapan : 67.60 %
=====

[KIRIM MESH] {"node":"Node-3","chipId":"3890256877","suhu":33,"kelembapan":68.2}

[TERIMA DARI] Node-2 (Node ID: 3155190338 | Chip ID: 3155190338)
Suhu : 32.10 °C
Kelembapan : 67.50 %
=====

Ln 1, Col 1 NodeMCU 1.0 (ESP-12E Module) on COM5 5:31 PM 10/6/2026

modul5_prak3 | Arduino IDE 2.3.10

File Edit Sketch Tools Help

NodeMCU 1.0 (ESP-12E Module)

modul5_prak3.ino

```
1 #include <painlessMesh.h>
2 #include <DHT.h>
3 #include <ArduinoJson.h>
4
5 #define MESH_PREFIX "LabIoTMesh"
6 #define MESH_PASSWORD "iotmeshpassword"
7 #define MESH_PORT 5555
8
9 #define DHTPIN 2
10 #define DHTTYPE DHT22
11
12 DHT dht(DHTPIN, DHTTYPE);
13
14 const char* nodeName = "Node-1";
15
16 Scheduler userscheduler;
17 painlessMesh mesh;
18
19 void sendMessage();
20
21 Task taskSendMessage(
22   TASK_SECOND, 3
23 )
```

Output Serial Monitor X

Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM3')

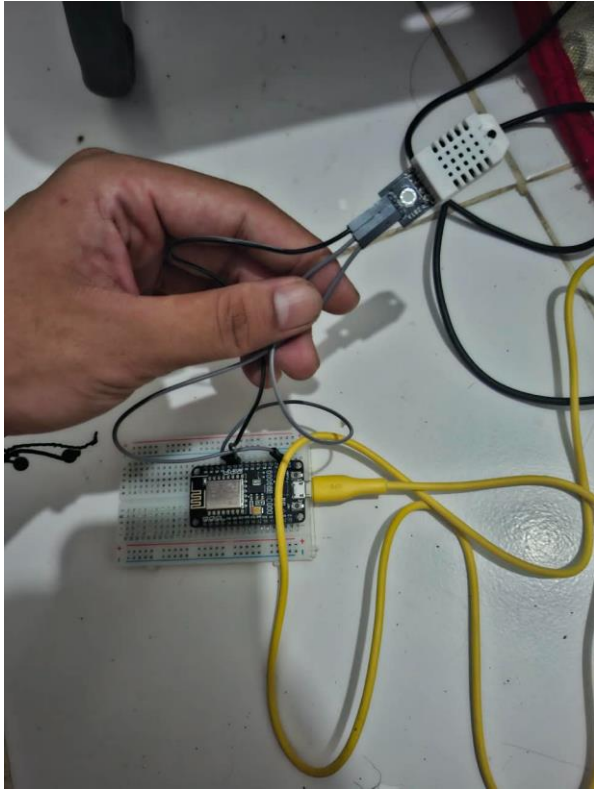
New Line 115200 baud

[TERIMA DARI] Node-2 (Node ID: 3155190338 | Chip ID: 3155190338)
Suhu : 32.20 °C
Kelembapan : 67.60 %
=====

[TERIMA DARI] Node-3 (Node ID: 3890256877 | Chip ID: 3890256877)
Suhu : 32.90 °C
Kelembapan : 70.20 %
=====

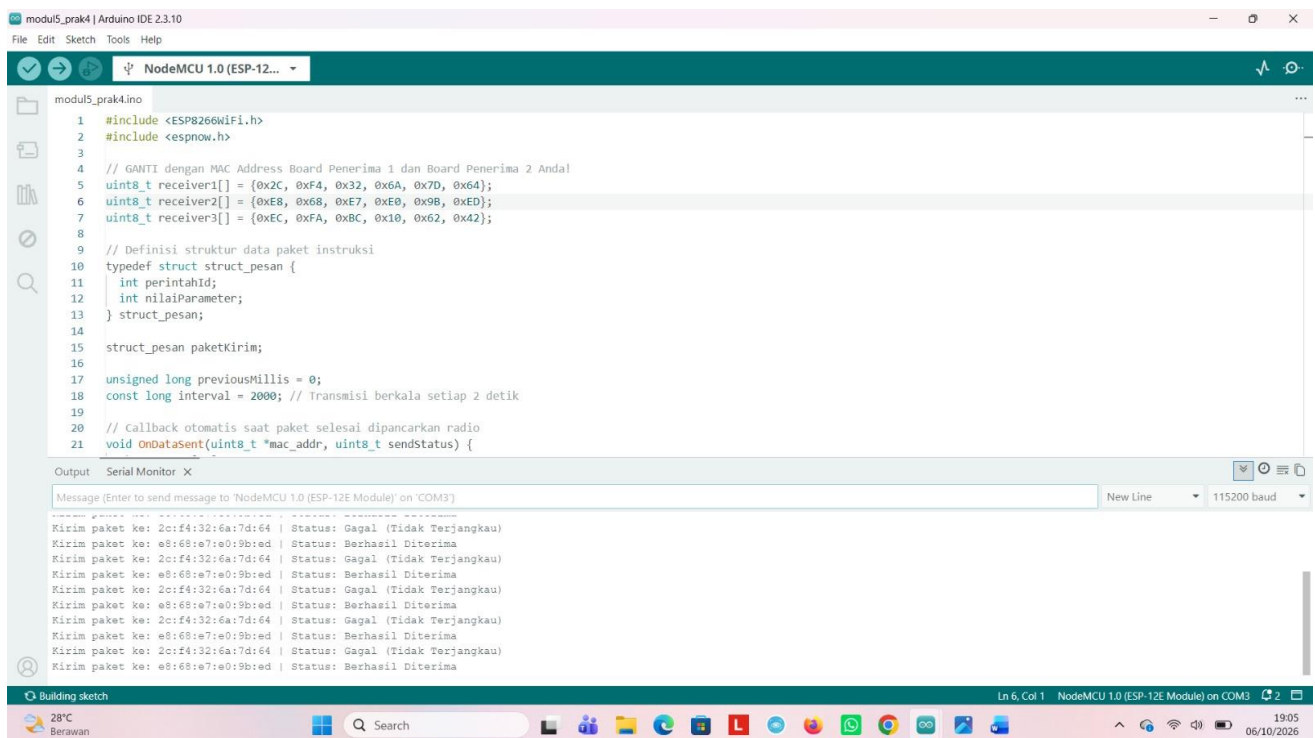
[KIRIM MESH] {"node":"Node-1","chipId":"845839716","suhu":33.2,"kelembapan":63.8}

indexing: 53/86 NodeMCU 1.0 (ESP-12E Module) on COM3 17:34 05/10/2026



Latihan

- **Latihan 1 (Eksperimen Deteksi ACK & Pemutusan Simpul - Praktikum 2):** Saat program Praktikum 2 (ESP-NOW One-to-Many) sedang berjalan normal dan mencetak status "Berhasil Diterima" (`sendStatus == 0`), lakukan eksperimen pemutusan simpul:
 - i. Cabut kabel USB salah satu board Penerima (misal NodeMCU 2) dari sumber daya.
 - ii. Amati perubahan log pada Serial Monitor NodeMCU Controller pengirim! Apakah status pengiriman untuk MAC address board yang dicabut berganti menjadi "Gagal (Tidak Terjangkau)"?
 - iii. Jelaskan bagaimana protokol ESP-NOW pada lapisan *Layer 2* MAC dapat mendeteksi kegagalan transmisi ke perangkat fisik tersebut meskipun sistem tidak menggunakan koneksi TCP!



```
modul5_prak4.ino
1 #include <ESP8266WiFi.h>
2 #include <espnow.h>
3
4 // GANTI dengan MAC Address Board Penerima 1 dan Board Penerima 2 Andai
5 uint8_t receiver1[] = {0x2C, 0xF4, 0x32, 0x6A, 0x7D, 0x64};
6 uint8_t receiver2[] = {0xE8, 0x68, 0xE7, 0xE0, 0x9B, 0xED};
7 uint8_t receiver3[] = {0xEC, 0xFA, 0xBC, 0x10, 0x62, 0x42};
8
9 // Definisi struktur data paket instruksi
10 typedef struct struct_pesan {
11     int perintahId;
12     int nilaiParameter;
13 } struct_pesan;
14
15 struct_pesan paketKirim;
16
17 unsigned long previousMillis = 0;
18 const long interval = 2000; // Transmisi berkala setiap 2 detik
19
20 // Callback otomatis saat paket selesai dipancarkan radio
21 void OnDataSent(uint8_t *mac_addr, uint8_t sendStatus) {
```

Output Serial Monitor X

Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM3')

Kirim paket ke: 2c:f4:32:6a:7d:64 | Status: Gagal (Tidak Terjangkau)

Kirim paket ke: e8:68:e7:e0:9b:ed | Status: Berhasil Diterima

Kirim paket ke: 2c:f4:32:6a:7d:64 | Status: Gagal (Tidak Terjangkau)

Kirim paket ke: e8:68:e7:e0:9b:ed | Status: Berhasil Diterima

Kirim paket ke: 2c:f4:32:6a:7d:64 | Status: Gagal (Tidak Terjangkau)

Kirim paket ke: e8:68:e7:e0:9b:ed | Status: Berhasil Diterima

Kirim paket ke: 2c:f4:32:6a:7d:64 | Status: Gagal (Tidak Terjangkau)

Kirim paket ke: e8:68:e7:e0:9b:ed | Status: Berhasil Diterima

Kirim paket ke: 2c:f4:32:6a:7d:64 | Status: Gagal (Tidak Terjangkau)

Kirim paket ke: e8:68:e7:e0:9b:ed | Status: Berhasil Diterima

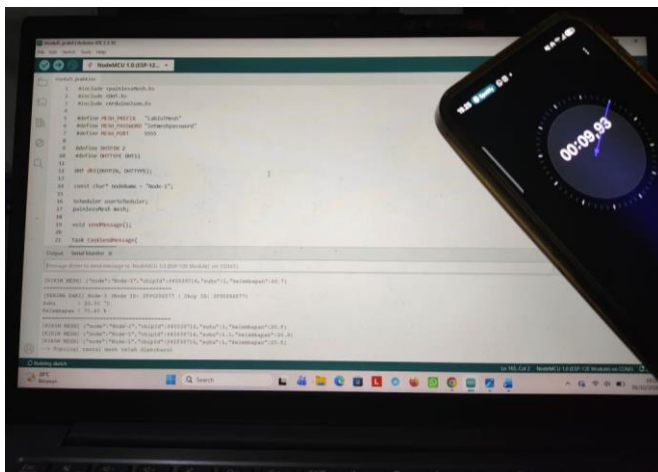
Ketika kabel USB NodeMCU 2 dicabut, NodeMCU tersebut kehilangan sumber daya sehingga radio Wi-Fi-nya juga mati. Controller tetap mencoba mengirim data ke MAC address NodeMCU 2, tetapi tidak mendapatkan ACK dari perangkat tersebut. Akibatnya, pengiriman dianggap gagal dan status yang ditampilkan menjadi: **Gagal (Tidak Terjangkau)**

- **Latihan 2 (Eksperimen Dinamika Topologi & Self-Healing - Praktikum 4):**

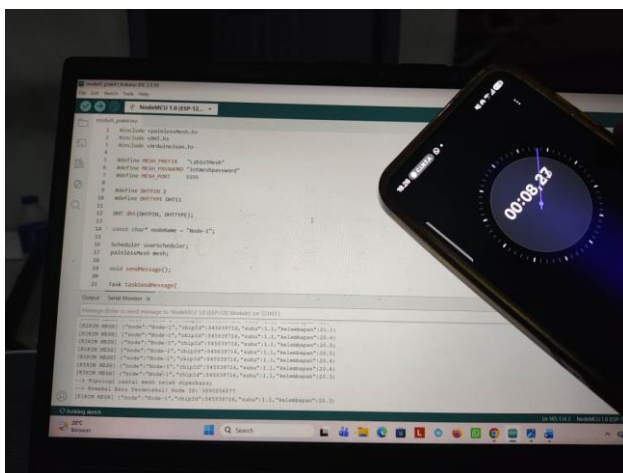
Jalankan jaringan painlessMesh pada Praktikum 4 hingga seluruh board saling terhubung dan mencetak pesan --> Koneksi Baru Terdeteksi! Node ID:

- i. Tekan tombol fisik **RST** atau cabut daya salah satu board Node. Amati Serial Monitor pada board lainnya yang masih menyala. Berapa detik waktu yang dibutuhkan oleh *callback* `changedConnectionCallback()` untuk mendeteksi bahwa salah satu simpul telah keluar dari rantai mesh?
- ii. Sambungkan kembali daya board tersebut. Berapa lama waktu yang dibutuhkan hingga node tersebut kembali terdaftar secara otonom ke dalam topologi mesh tanpa Anda mengonfigurasi ulang jaringan? Catat hasil observasi waktu tersebut!

Pencabutan:



Penyolokan:



Rata rata waktu yang dibutuhkan agar hingga node tersebut kembali terdaftar secara otonom ke dalam topologi mesh tanpa Anda mengonfigurasi ulang jaringan memiliki waktu hamper 9 detik

- Latihan 3 (Eksperimen Modifikasi Filter Telemetry & Ambang Batas JSON - Praktikum 4):** Pada Praktikum 4, setiap node saat ini mencetak seluruh data telemetry dari semua pengirim tanpa penyaringan.
 - Modifikasilah fungsi `receivedCallback` pada salah satu board agar bertindak sebagai **Pengawas Kondisi Suhu Kritis**: program **HANYA** mencetak kartu telemetry penuh ke Serial Monitor jika suhu yang dilaporkan oleh node tetangga bernilai **di atas 32.0 °C** (lakukan pengujian dengan mendekatkan jari hangat atau meniup sensor DHT pada node pengirim).
 - Jika suhu bernilai normal (≤ 32.0 °C), program cukup mencetak status singkat satu baris: `[NORMAL] Telemetry dari Node <chipId> aman.`
 - Tuliskan blok fungsi `receivedCallback(uint32_t from, String &msg)` hasil modifikasi Anda!

The screenshot shows the Arduino IDE interface with the file `modul5_prak4.ino` open. The code is for a NodeMCU 1.0 (ESP-12E) module. It includes a `void loop()` function that calls `mesh.update()`. The Serial Monitor is open, showing the output of the program. The output includes a message from Node-3 with critical temperature and humidity data, and a message from Node-1 with normal temperature and humidity data.

```

200
201
202 // Pesan awal
203 Serial.printf(
204   "Mesh Node [%s] Berjalan. Menunggu pembentukan topologi...\n",
205   nodeName
206 );
207
208 }
209
210
211 // =====
212 // LOOP
213 // =====
214 void loop() {
215
216   // Wajib dipanggil untuk menjalankan mesh
217   mesh.update();
218
219 }
220

```

Serial Monitor Output:

```

=====
[SUHU KRITIS] Telemetry dari Node-3
Node ID   : 3890256877
Chip ID   : 3890256877
Suhu      : 32.80 °C
Kelembapan : 71.20 %
Status    : SUHU DI ATAS AMBANG BATASI!
=====
[NORMAL] Telemetry dari Node 3155190338 aman.
[KIRIM MESH] ("node":"Node-1","chipId":"845839716","suhu":32.2,"kelembapan":70.4)

```

Blok modifikasi

```

1 // Callback otomatis saat menerima pesan dari node manapun di jaringan mesh
2 void receivedCallback(uint32_t from, String &msg) {
3   // Parsing pesan JSON yang diterima
4   StaticJsonDocument<200> doc;
5   DeserializationError error = deserializeJson(doc, msg);
6
7   if (!error) {
8     const char* sender = doc["node"];
9     uint32_t chipId = doc["chipId"];
10    float suhu = doc["suhu"];
11    float kelembapan = doc["kelembapan"];
12
13    // =====
14    // FILTER SUHU KRITIS
15    // Hanya tampilkan telemetry lengkap jika suhu > 32.0 °C
16    // =====
17    if (suhu > 32.0) {
18      Serial.println("=====");
19      Serial.printf("[SUHU KRITIS] Telemetry dari %s\n", sender);
20      Serial.printf("Node ID : %u\n", from);
21      Serial.printf("Chip ID : %u\n", chipId);
22      Serial.printf("Suhu : %.2f °C\n", suhu);
23      Serial.printf("Kelembapan : %.2f %\n", kelembapan);
24      Serial.printf("Status : SUHU DI ATAS AMBANG BATASI!");
25      Serial.println("=====");
26    } else {
27      // Jika suhu normal, hanya tampilkan satu baris
28      Serial.printf("[NORMAL] Telemetry dari Node %u aman.\n", chipId);
29    }
30  } else {
31    // Jika data yang diterima bukan JSON yang valid
32    Serial.printf("[TERIMA DATA MENYAH DARI %u]: %s\n", from, msg.c_str());
33  }
34  ..
35 }

```